

Zakura sendrawtransaction Retry Channel Poisoning Risk Report

Summary

sendrawtransaction sends a deserialized transaction into a 20-item retry channel before full mempool validation finishes. Consensus-invalid transactions can therefore consume channel work and create lag even though the RPC later rejects them; final eviction of a valid retry transaction remains an evidence boundary.

Mitigating Factors:

- The default RPC deployment requires a privileged cookie, reducing anonymous access.
- Users can manually resubmit a transaction; the reviewed evidence does not establish fund loss or a consensus split.
- Confirmed channel lag does not by itself prove that the downstream IndexMap was full or that a legitimate transaction was permanently lost.
- RPC source restrictions and request-rate limits can reduce abuse while queue semantics are corrected.

Aggravating Factors:

- `queue_sender.send()` runs before the full mempool result is known, and later failure does not retract the item.
- The channel capacity is only 20, so a small burst can create lag and skipped-channel notifications.
- Permanent and temporary validation failures are not cleanly separated before retry admission.
- If RPC is exposed without authentication, the attacker population expands from trusted administrators to any reachable remote user.

Vulnerability Details:

Status, Product, and Version

- Record status: PUBLISHED
- Product and version: Zakura / zakurad / 1.2.0
- Weakness classification: CWE-770: Allocation of Resources Without Limits or Throttling; CWE-400: Uncontrolled Resource Consumption

Cross-Validation Conclusion

Early enqueue and retry-channel pressure are valid. The strongest established consequence is wasted retry work and channel lag. A claim that a valid transaction is finally evicted and loses relay requires a controlled end-to-end comparison and should not be treated as proven here.

Trigger Conditions

- The caller can access sendrawtransaction; default deployments require the RPC cookie.
- Submitted bytes deserialize as a transaction but fail later consensus or mempool checks.
- Enough requests arrive to fill or overrun the 20-item retry channel.
- The retry worker processes attacker-supplied entries before classifying permanent failure.

Normal and Abnormal Behavior

Normal behavior

Only transactions that need a legitimate temporary retry condition enter the retry structure, and permanent validation failures are discarded immediately.

Abnormal behavior

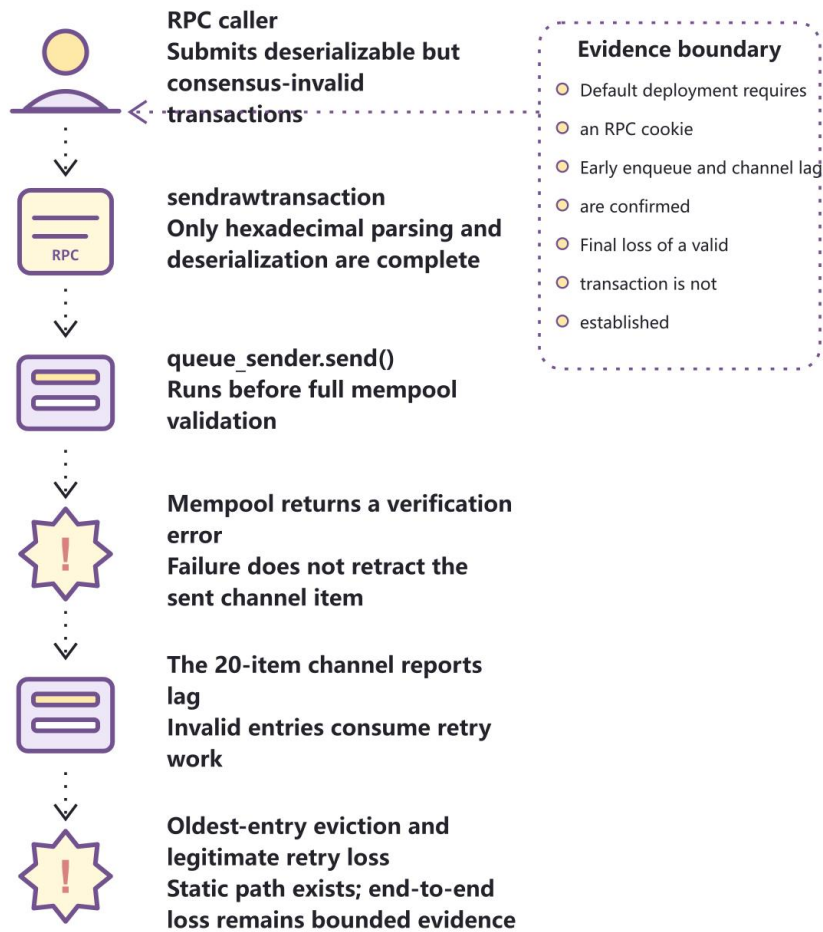
A transaction occupies channel work before validation completes; a later permanent failure returns an RPC error but does not undo the earlier enqueue.

Full Call Chain Diagram

The complete call chain is shown below. The dashed box states the deployment condition, version boundary, or evidence boundary that controls the conclusion.

Retry-Channel Poisoning Call Chain

A transaction enters the channel before full validation completes



Technical Details:

Root cause 1: retry admission precedes full validation

```
let transaction = deserialize(raw_transaction)?;  
queue_sender.send(transaction.clone())?;  
let result = mempool.call(transaction).await;
```

The ordering means a rejected transaction can still reach the retry consumer.

Root cause 2: the channel has a small shared capacity

```
const CHANNEL_AND_QUEUE_CAPACITY: usize = 20;  
let (sender, receiver) = broadcast::channel(CHANNEL_AND_QUEUE_CAPACITY);
```

Root cause 3: oldest-entry removal can affect unrelated work

When the downstream retry structure reaches capacity, oldest-first removal does not inherently distinguish attacker-generated permanent failures from legitimate temporary retries.

Minimal Reproduction Logic (Isolated Test Environment Only)

Use an isolated node and a bounded set of deserializable but consensus-invalid transactions. Confirm RPC rejection and channel lag, then run a separate controlled valid-retry case before making any claim about final eviction.

```
repeat 25 times:  
    call sendrawtransaction(<deserializable-consensus-invalid-transaction>)  
observe: RPC verification errors and retry-channel lag metrics
```

Security Impact

- Invalid transactions can consume retry-channel capacity and repeated validation work.
- Legitimate retry processing may be delayed under pressure.
- Permanent loss of a valid transaction is not established by channel-lag evidence alone.

Long-Term Remediation

- Enqueue only after full validation, or enqueue only explicitly classified temporary failures.
- Never retry permanent consensus failures.
- Apply per-caller quotas and a failure-aware eviction policy.
- Expose separate metrics for channel lag, downstream queue occupancy, failure class, and legitimate retry completion.

Recommended Regression Tests:

- Verify that a permanent validation failure never enters the retry consumer.
- Fill the channel with invalid submissions and confirm a legitimate temporary retry remains represented.
- Compare exposed-authentication and default-cookie configurations.
- Confirm that retry metrics distinguish channel lag from downstream eviction and final transaction outcome.